



26. Mai 2026

Übungen zur Vorlesung Service-basierte Komponenten-Architekturen

SS 2026

Übungsblatt Nr. 5

(Abgabe bis: Dienstag, den 10. Juni 2025, **12:00 Uhr**. Auch eine spätere Präsentation einer erweiterten *oder* reiferen Version der Lösung wäre möglich. Auch eine Vorab-Präsentation einer Lösung wäre – nach Absprache - denkbar)

Aufgabe 1 (Teilprojekt „Entwicklung einer Choreographie von Microservices“)

Die Geschäftsleitung der Firma WirSchiffenDas GmbH ist wenig begeistert von der starren und wenig einsichtigen Haltung der Abteilung Technik zum Thema Microservices und verlangt daher, ohne Abstimmung mit dem Abteilungsleiter, einen ersten Proof-Of-Concept von *ihnen*! Man ist dabei interessiert, die kritische Analyse-Komponente innerhalb der Komponente „Manufacturing Products“ mittels Microservices zu optimieren. Man ist dabei angetan von der Idee des Ingenieurs, der im Rahmen einer ersten Anforderungsanalyse interviewt wurde (vgl. Unterlage Case Study, Folie 9):

„Die Analyse-Komponente für Komponenten ist deswegen so komplex, weil für jedes optionale Equipment ein eigenständiger und abgeschlossener Algorithmus zur Validierung und Simulation der Konfiguration durchgeführt werden muss. Aktuell sind diese Algorithmen in einer Klasse integriert. Ob man das nicht mal auftrennen könnte, indem man die Algorithmen in einer Choreographie nebenläufig abarbeitet? Die übrigen Komponenten sollte also responsive bleiben! Möchte auch gerne die aktuellen Status der einzelnen Algorithmen sehen können, das ist in der aktuellen Implementierung nicht möglich. Auch ein Retry eines einzelnen Algorithmus sollte möglich sein. (...)“

Natürlich soll die Analyse-Komponente nicht originalgetreu nach-implementiert werden, aber die wichtigsten Abläufe sollen simuliert werden. Gehen sie wie folgt vor:

- Simulieren Sie die Eingabe einer Konfiguration des Optional Equipments für eine Diesel Engine. Dies kann eine einfache Eingabe-Maske sein, die über einen separaten Microservice bereitgestellt werden *kann*. Als Eingabe-Werte können Sie die möglichen Werte aus der Tabelle auf Folie 9 verwenden. Können Sie keine alternativen Eingabewerte identifizieren, so können sie Dummy-Werte annehmen.
- Eine Konfiguration soll über einen Microservice persistent verwaltet werden können.

- Die eigentliche Analyse soll über einen entsprechenden Button oder einem sonstigen Interaktionselement ausgeführt werden können. Durch diese Interaktion wird ein bestimmter „Anker“-Algorithmus ausgeführt.
- Wie von dem Ingenieur vorgeschlagen, *sollten* die Algorithmen auf *choreographierte* Microservices aufgeteilt werden, die sich nacheinander in einer Sequenz aufrufen können. Auch eine *parallele* Ausführung von Microservices im Rahmen einer Choreographie wäre denkbar. Die Algorithmen können über entsprechende REST-Endpoints bei den Microservices aufgerufen werden. Sie können die Durchläufe der Algorithmen simulieren, indem sie z.B. den Thread der jeweiligen Implementierung für eine Zeit (z.B. 20 sec) anhalten (suspendieren). Nach Beendigung eines Algorithmus übermittelt der Microservice das Ergebnis *proaktiv*.
- Sie können die Algorithmen nach einer eigenen Systematik clustern (Beispiel: ein Algorithmus zur Validierung der Flüssigkeits-Konfiguration, also der Optional Equipments „Oil System“ und „Fuel System“). Somit sollten sie *mindestens* 4 Algorithmen bzw. eigenständige Microservices bereitstellen.
- Der Algorithmus zur Analyse des Engine Management Systems baut *je nach Konfiguration* auf den vorherigen Ergebnissen der anderen Algorithmen auf. Bitte diese Rahmenbedingung *exemplarisch* bei der Implementierung berücksichtigen.
- Sie können ansonsten davon ausgehen, dass die Algorithmen unabhängig voneinander agieren, diese sind also nicht auf das Vorergebnis eines Algorithmus angewiesen. Jeder Algorithmus nimmt sich aus dem Input, die gegebene Konfiguration des Optional Equipments, seinen Anteil, den er berechnen muss. Zu jedem Optional Equipment soll nach Beendigung der Analyse ein Ergebnis („Result“) angezeigt werden. Hier können Sie z.B. die Werte „ok“ oder „failed“ verwenden. Auch das Gesamtergebnis sollte über alle Optional Equipments angezeigt werden.
- Wie in der Anforderungsbeschreibung des Ingenieurs beschrieben, soll der Bearbeitungs-Status der Algorithmen in der Benutzeroberfläche der Anwendung stets zu entnehmen werden. Als Status können sie *pro* Algorithmus *zumindest* folgende Werte annehmen: „running“, „failed“, „ready“. Die Microservice übermitteln die Zustände *proaktiv* über einen designierten Endpunkt.
- Beachten sie zudem den Fall, dass Microservices nicht erreichbar sind. Verwenden sie das Pattern Circuit Breaker (Kapitel 4), um die Widerstandsfähigkeit der gesamten Anwendung zu garantieren. Der Ausfall eines Microservices und die damit verbundenen Auswirkungen sollten sie natürlich entsprechend simulieren.
- Machen sie sich Gedanken über die Gestaltung der Schnittstellen der einzelnen Microservices; diese sollten als REST-Endpoints entwickelt werden.

a.) Plattform zur Implementierung der Microservices

Ich empfehle die Entwicklung der Microservice-basierten Architektur bzw. der einzelnen Microservices mit der Plattform „Spring Boot“ (vgl. Kapitel 2 sowie Übung Nr. 3).

Optional können sie überlegen, die Microservices mit einer *alternativen Technologie* wie z.B. Node.js oder dem Django REST Framework zu implementieren. Auch die Verwendung der Plattform Docker für eine Containerisierung der Microservices wäre denkbar.

b.) Modellierung und Dokumentation der Software-Architektur

Modellieren Sie die Baustein-Sicht ihrer resultierenden Software-Architektur entsprechend. Dokumentieren Sie die wichtigsten Entwurfsentscheidungen anhand des arc42-Templates (vgl. Kapitel 3).

c.) Entwicklung einer Benutzeroberfläche

Für die Entwicklung der Benutzeroberfläche *können* Sie ein beliebiges Framework verwenden, so z.B. Angular, JavaFX oder Vaadin. Auch ein Command Line Interface (CLI) oder der Einsatz eines Test-Tools wie Postman (vgl. Übung Nr. 3-2) wäre denkbar für einen *ersten* Prototyp (vgl. Quellen aus Übung Nr. 2).

ToDo in der Übungsstunde (26.5.26) (Bearbeitungszeit: ca. 30 min.): Entwerfen Sie eine erste High-Level-Architektur sowie einen ersten „Papier-Prototyp“ der Benutzeroberfläche. Auch einen ersten Entwurf der REST-Endpoints sollten sie ableiten.

Ausblick für dieses Teilprojekt im Kontext der Fallstudie „WirSchiffenDas“ (nicht zwingend für diese Übung!!), falls Sie dies als Semesterprojekt umsetzen möchten:

Für eine vollständige und solide Implementierung sollten sie mindestens eine von den drei Technologien bzw. technischen Anforderungen umsetzen:

MS_TA1: Verwendung des Framework Apache Kafka für eine Integration der Microservices zur Übermittlung insbesondere der „Bearbeitungs-Status-Nachrichten“.

MS_TA2: Verwendung der Technologien Docker und Docker Compose zur Bereitstellung und Komposition der Microservices in einsetzbare Docker Container.

MS_TA3: Verwendung des Spring-Netflix-Stacks, um wichtige Pattern wie Service Registry oder Circuit Breaker sinnvoll zu integrieren (Alternative: Resilience4j)

Weitere mögliche Teilprojekte, die im Rahmen der Fallstudie „WirSchiffenDas“ als Semesterprojekt alternativ umgesetzt werden können:

- Orchestration von (Micro-)Services für den Prozess „Order Fullfillment“ mit Camunda (inklusive einer *einheitlichen* Web-UI)
 - Weitere Informationen dazu im Rahmen der Übung Nr. 6 sowie in Kapitel 6
 - Auch hier sollten mindestens eine der technischen Anforderungen *MS_TA2* und *MS_TA3* umgesetzt werden
- Orchestration von (Micro-)Services für den Prozess „Order Fullfillment“ mit MCP und einer LLM (inklusive einer CLI / Chat)
 - Weitere Informationen dazu im Rahmen der Übung Nr. 6 sowie in Kapitel 6
 - Auch hier sollten mindestens eine der technischen Anforderungen *MS_TA2* und *MS_TA3* umgesetzt werden
- Implementierung des Prozesses „Order Fullfillment“ durch die Integration der einzelnen UIs der (Micro-)Services in ein UI-Frame (Micro-Frontend, siehe Kapitel 4, Fol. 14, Option Nr. 3 (ganz rechts)).
 - Auch hier sollten mindestens eine der technischen Anforderungen *MS_TA2* und *MS_TA3* umgesetzt werden
 - Ansonsten keine weiteren Informationen zu diesem möglichen Teilprojekt

Quellen für die Entwicklung eines Microservice (*eigenständige* Einarbeitung!):

REST-basierter (Micro-)Service mit SpringBoot:

<https://spring.io/guides/gs/rest-service/>

Als Grundlage für die Entwicklung eines Microservice mit den Technologien SpringBoot und Docker steht Ihnen auch ein initiales Repository zur Verfügung (vgl. Aufgabe 3-2):

<https://git.fslab.de/salda2m/microservice#>

REST-basierter (Micro-)Service mit Node.js und Express:

<https://stackabuse.com/building-a-rest-api-with-node-and-express/>

REST-basierter (Micro-)Service mit Django:

<https://www.django-rest-framework.org/tutorial/quickstart/>

REST-basierter (Micro-)Service mit Flask:

<https://flask-restful.readthedocs.io/en/latest/>

Quellen für Docker:

Entwicklung einer Docker-Anwendung auf Basis von Spring Boot:

<https://spring.io/guides/topicals/spring-boot-docker>

Entwicklung einer Docker-Anwendung auf Basis von Node.js:

https://docs.docker.com/get-started/02_our_app/

Quellen für die Entwicklung einer Web-basierten Oberfläche mit Angular:

Dokumentiertes Framework zur Entwicklung einer Anwendung basierend auf Angular und Node.js („MEAN“):

<https://github.com/aldaGit/iar-framework>

(c/o Luca Ringhausen, Prof. Dr. Sascha Alda)

Zugehöriges Video-Tutorial (in englischer Sprache, ca. 2 Stunden):

<https://www.youtube.com/watch?v=IOQhgrugBy4>

(c/o Prof. Dr. Sascha Alda)

Quellen für die Entwicklung einer Web-basierten Oberfläche mit Vaadin:

Dokumentiertes Framework zur Entwicklung einer Anwendung basierend auf Vaadin 24:

<https://github.com/aldaGit/carlook>

(c/o Prof. Dr. Sascha Alda)

Zugehöriges Video-Tutorial (in deutscher Sprache, 7 Teile, ca. 2,5 Stunden):

<https://github.com/aldaGit/carlook#links-zu-video-tutorien>

(c/o Prof. Dr. Sascha Alda)

(Last Access all URLs: 26.5.2026)